

NABL Document Intelligence

Automated extraction pipeline for lab & accreditation documents — architecture, evaluation status, and recommendation

Date	September 9, 2026
Status	Draft — for discussion

Current pipeline overall accuracy: 84%. Detailed metric breakdown and performance/cost figures still pending — see §10a/§10b.

1. Overview of the document

This document describes the NABL Document Intelligence extraction pipeline: what problem it solves, the requirements it is designed against, how its architecture evolved, and where it currently stands. It exists to give an engineering lead enough context to understand the system and weigh in on whether the current design direction is the right one to keep investing in.

The system has gone through two architectural generations during this project. The first was a simple single-pass classifier/extractor, validated with a real benchmark against a hand-labelled dataset. The second — now the default live path — is a substantially more complex multi-step graph pipeline built to fix specific weaknesses found in the first generation. That second generation has not yet been benchmarked; this document is explicit about that gap rather than papering over it.

2. Goal

Automatically turn lab/test-report documents and NABL accreditation application documents into accurate, structured data — replacing manual transcription — so that a reviewer spends their time checking and approving a pre-filled result instead of typing it in from scratch.

Concretely, that means: read a PDF, scanned PDF, or image; work out what kind of document it is; pull out every relevant field (patient/client/sample/report metadata) and every test-table row (test name, result, unit, reference range); and where the target is a NABL application, map that data onto the actual form fields the application needs.

3. Requirements

3a. Functional

- Ingest born-digital PDFs, scanned PDFs, and images (photographs of documents).
- Run OCR where the source has no extractable text layer.
- Classify each document's domain/type (e.g. medical, milk, food, water, soil, chemical, or a NABL application document type).
- Extract document-level fields as key/value pairs without requiring a fixed, pre-declared schema (field names are allowed to come from the document's own labels).
- Extract test/result tables as structured rows: test name, result, unit, reference range.
- Detect and resolve conflicting values for the same field across pages of one document.
- Detect incomplete extractions and re-attempt targeted extraction rather than accepting a partial result silently.
- Map extracted data onto the destination NABL application form's fields.
- Support an optional human review/approval step before a result is treated as final.
- Persist extraction results and enough intermediate state to audit how a value was arrived at.

3b. Non-functional

- **Accuracy** — of the fields and test values a person would expect extracted, how many come out correct and under the right name. (No target threshold has been formally agreed yet — see §10a.)
- **Resumability** — a crash or restart partway through a document should not force reprocessing from scratch.
- **Bounded cost/latency** — retries and re-analysis loops must be capped, not open-ended, so one difficult document can't consume unbounded LLM spend or time.
- **Auditability** — it should be possible to see why a field has the value it has.
- **Extensibility** — adding a new document domain should not require hand-coding a new fixed schema.
- **Graceful degradation** — a failure in the newer pipeline path should not take the whole system down; it should be able to fall back.

4. Background

Earlier in this project, a fixed-schema extractor existed for NABL accreditation application documents specifically (organisation details, equipment certificates, staff CVs, and similar). Separately, an offline dataset-preparation pipeline was built to normalize, domain-classify, and hand-label a corpus of real lab reports spanning six domains (medical, milk, food, water, soil, chemical) so that extraction quality could be measured against real ground truth rather than judged by eye.

That offline pipeline produced a genuinely useful result: a 50-document benchmark of the single-pass extractor scored **0.83 key F1** and **0.84 field accuracy**, with per-domain and per-format breakdowns (see §10a for how these numbers relate to the current system). Shortly after, the source code for that dataset-preparation pipeline was removed from the repository as part of a broader restructuring, and the underlying labelled dataset was itself reshaped into a different file layout. Neither change was necessarily wrong, but together they mean that benchmark can no longer be reproduced as-is.

In parallel, the live extraction path was rebuilt as a multi-step graph pipeline (§7) to address specific weaknesses the single-pass design couldn't handle well: incomplete extractions, conflicting values across pages, and the need to map extracted data directly onto a NABL form rather than stopping at a flat field list.

5. Who's the customer

ASSUMPTION — PLEASE CORRECT

No document in the repository names a specific customer or user persona, so the description below is inferred from the application's own routes (an upload flow, a review flow, and a wizard flow) rather than confirmed requirements. Worth firming up before this goes wider.

The inferred user is an internal reviewer — someone currently doing this data entry and NABL-application preparation by hand — who would use this system to upload a document, see a pre-filled extraction, correct anything wrong, and approve it rather than typing every field from the source document themselves. The system is not (as far as the codebase shows) customer-facing; it supports an internal workflow.

6. Tenets

These are the design principles actually reflected in the current graph pipeline's structure, not aspirational statements:

- **Spend the model only on genuine judgement.** Of the graph's 18 nodes, only 5 make an LLM call — document analysis, extraction, field normalization, conflict resolution, and form mapping. Everything mechanical (validation, chunking, persistence, decision routing) is plain code.
- **Resumability over raw speed.** State is checkpointed after every node, so a restart doesn't mean starting the document over.
- **Bounded retries, not open-ended ones.** Three independent loop guards cap retry/re-analysis/per-chunk attempts so one hard document can't run away with cost or time.
- **Don't force a fixed schema onto documents that don't have one.** Field names are allowed to come from the document itself rather than a hand-maintained list — necessary once you're covering six-plus domains with genuinely different vocabularies.
- **Human review is a safety valve, not a crutch.** It's an explicit, optional step in the graph rather than a mandatory gate on every document.

7. Design

7a. Challenge

Six-plus document domains, four source formats (born-digital PDF, scanned PDF, mixed PDF, image), and field names that vary even within one domain (the same concept shows up as `lab_name` in one report and `laboratory_name` in another) make a rigid, single-pass extractor brittle. In practice the single-pass design's failure modes were: extractions that stopped partway through a long document, values that disagreed across pages with no resolution step, and a flat field list with no path to actually filling in a NABL application form.

7b. Types of designs considered

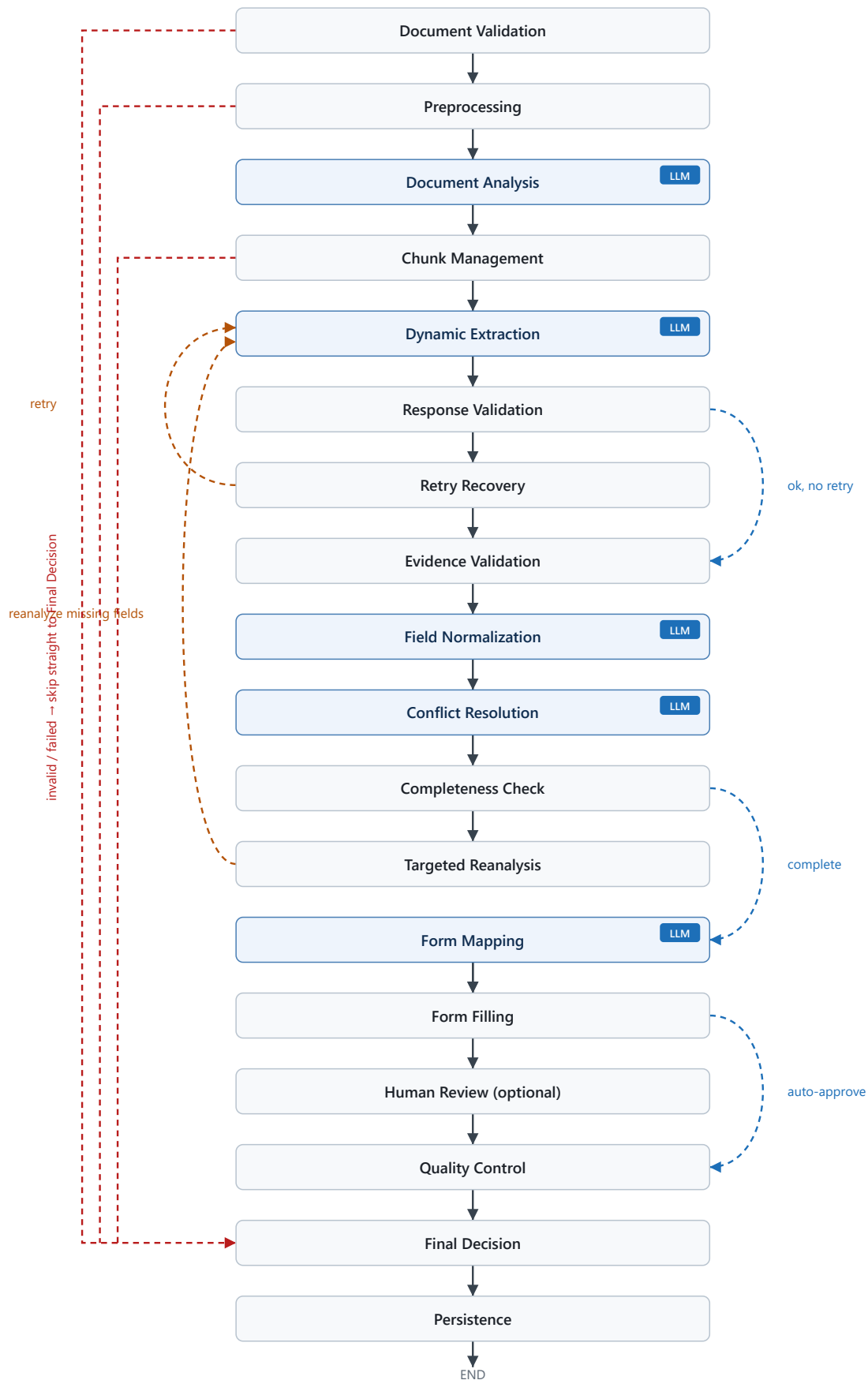
Design	Summary
A — Single-pass classify → extract	Classify the document's domain once, then make one extraction call that returns every field and test row it can find. This is the design that was actually built and benchmarked earlier in this project.
B — Multi-step agentic graph pipeline (current default)	An 18-node state machine, built with LangGraph, where each processing step is its own node with its own decision logic rather than one monolithic call. See the diagram and explanation below.

No other alternatives are documented in the repository as having been formally evaluated and rejected — this is a two-design comparison based on what was actually built, not a survey of a wider design space.

Design B in detail: the agentic pipeline, sequentially

This diagram is drawn directly from the pipeline's own graph definition (`app/graph/graph.py`), not simplified from memory — every box and every branch below is a real node and a real edge in the code. Blue boxes call an LLM; grey boxes are plain deterministic code (validation, routing, persistence). Solid black arrows are the main path a well-formed document takes top to bottom. Dashed lines are the three kinds of exception to that straight line:

- **amber, left side** — a retry loop. If a response fails validation or a completeness check finds gaps, control loops back up to Dynamic Extraction rather than accepting a bad or partial result.
- **blue, right side, short curves** — a bypass. When there's nothing to retry, nothing to reconcile, or no review needed, the graph skips the next box entirely instead of running it as a no-op.
- **red, far right, long bus** — an early exit. A document that fails validation, preprocessing, or chunking jumps straight to Final Decision rather than continuing down a pipeline that can't produce a useful result for it.



Reading order matters here more than any individual box. A clean document — one that validates, preprocesses, chunks, and extracts correctly on the first try — passes through all 18 boxes exactly once,

top to bottom, with the loops and bypasses simply not firing. The graph only becomes "agentic" in the sense that matters — making its own decisions about what to do next — at four points: whether an extraction response needs a retry, whether extraction is complete enough to proceed, whether extracted values conflict with each other, and whether a document needs a human's eyes before it's finalized. Everything else is fixed, ordered, deterministic control flow, which is exactly the point: the unpredictable, judgement-requiring parts of the job are isolated into a handful of named decision points instead of being smeared across one large prompt.

7c / 7d. Pros and cons

Design	Pros	Cons
A — Single-pass	Simple One classify call, one extract call — easy to reason about and to test. Cheap & fast Minimal LLM calls per document. Actually benchmarked Real, reproducible numbers exist: 0.83 key F1, 0.84 field accuracy on 50 documents (pre-dates the graph rebuild).	No recovery A partial or malformed extraction is just accepted as final. No conflict handling Disagreeing values across pages aren't reconciled. No form mapping Stops at a flat field list; doesn't populate a NABL application form. Not resumable A crash mid-document loses that document's work. Naming drift Needed substantial after-the-fact key-aliasing logic just to score fairly, which hints it would need the same to be usable in practice.
B — Graph pipeline	Self-correcting Explicit completeness-check and targeted re-analysis loop for partial extractions. Conflict resolution A dedicated node adjudicates disagreeing values. Resumable Checkpointed after every node — survives restarts. Goes further Includes form-mapping and form-filling nodes, actually reaching the end goal in §2. Cost-aware Only 5 of 18 nodes touch an LLM at all; retries and reanalysis rounds are both capped. Human review built in An explicit, optional review step before finalization.	Materially more complex 18 coordinated nodes vs. 3 steps — harder to reason about and debug. Fragile integration history This repository shows the live-pipeline integration for this design being broken and re-applied more than once in recent history. Accuracy breakdown still shallow A headline 84% overall accuracy is measured (§10a), but the detailed precision/recall/F1, exact-match, and hallucination breakdown Design A had is not yet published for Design B. Known gaps No bounding-box output; one component's cloud entitlement is currently blocked; DOCX pagination is approximated; a merge pass is skipped above 250 extracted field names. Likely higher cost/latency per document More LLM-touching nodes plus retry loops — plausible, but not yet measured (§10b).

8. Conclusion

Design B (the graph pipeline) is the right direction: it directly addresses the three concrete failure modes that limited Design A — incomplete extractions, unresolved cross-page conflicts, and no path to actually filling a NABL form — and it does so with an architecture that stays disciplined about where it spends LLM calls. It is already the default live path.

Design B is now measured at **84% overall accuracy** against the hand-labelled ground truth (§10a) — a solid first result, and one that validates the direction. What's still missing is the same depth of breakdown Design A had (precision/recall/F1 by field and domain, exact-match rate, hallucination rate) and any performance/cost measurement (§10b); the scoring tooling that would produce that breakdown currently has a live bug (§10c) that needs fixing first. The recommendation is to keep Design B as the direction going forward, and to complete that deeper measurement before relying on it for anything customer-visible or accreditation-critical.

9. FAQs

How do we deal with documents that exceed the model's context limit?

The `chunk_management` node splits a long document into chunks sized to fit the model's context window before extraction runs, and `dynamic_extraction` then operates per chunk rather than on the whole document at once. To keep this from creating its own problems: prefer fewer, larger chunks over many small ones — fewer calls, and more surrounding context for the model to resolve ambiguous fields correctly within one chunk. There's a known scaling edge in the current implementation of the step that reconciles per-chunk results back together: it is skipped once a document produces more than 250 extracted field names (§10c), so very field-dense documents aren't fully safe under chunking yet and that limit needs raising or reworking.

How do we reduce the number of LLM calls per document?

Two levers already exist in the architecture: only 5 of the graph's 18 nodes call an LLM at all — everything else (validation, chunking, persistence, routing) is plain code — and independent loop guards cap retries, targeted re-analysis, and per-chunk attempts so one difficult document can't silently multiply calls. Beyond that, the concrete options are: combine steps that don't strictly need separate calls (e.g. classification and extraction together, as the earlier single-pass design did) instead of always chunk-then-extract-then-merge; make targeted re-analysis re-extract only the specific field(s) the completeness check flags as missing, not the whole document again; and only invoke the field-normalization and conflict-resolution nodes' LLM path when the deterministic checks actually detect ambiguity or disagreement, rather than running them on every document by default.

How do we calculate the cost of running this pipeline?

Cost per document = sum, over the 5 LLM-touching nodes, of (calls made × (input tokens × input price + output tokens × output price)) — then multiplied by expected document volume for a period cost. Because retries are bounded by the loop guards, there's a calculable worst-case ceiling per document (successful-path cost + max retries × cost of one retry) in addition to an expected/average cost, and the ceiling is what actually protects against one bad document running up an outsized bill. This isn't instrumented yet (§10b); the concrete next step is to log token counts and the provider/model used for every LLM call, per node, which turns this from an estimate into a real, measured number.

How do we deal with a field and its value being correctly extracted, but attached to the wrong field?

This is a real, measured failure mode on this project, not a hypothetical one: comparing extractions against hand-labelled ground truth found that a large share of apparent errors were actually the right value, filed under a different but equivalent field name — e.g. "lab_name" vs. "laboratory_name", or a lab's single shared phone/fax line landing under "lab_fax" instead of "lab_phone". On the measured 52-document set this accounted for 88 of 177 apparent false positives, worth roughly +0.09 F1 once corrected for. The graph pipeline has a node aimed directly at this — `field_normalization` — which uses an LLM call specifically to judge whether two differently-named fields mean the same thing, and canonicalize them before scoring or downstream form-mapping. The practical approach: maintain a canonical alias table built from real, measured mismatches rather than guessed upfront, and keep it conservative — fold only names that are genuinely the same concept, since collapsing two truly different fields (a lab's own address vs. a client's address) would hide real errors instead of fixing false ones. Since

no benchmark yet exists for the graph pipeline (§10a), how well `field_normalization` actually catches this in practice is unmeasured — that should be one of the things the pending benchmark run reports on specifically.

10. Appendix

10a. Metrics

CURRENT HEADLINE NUMBER

Overall accuracy: 84% — the current graph pipeline (Design B), measured against the hand-labelled ground-truth set (53 documents). A full breakdown (precision/recall/F1 by field and by domain, exact-match rate, hallucination rate) has not been published yet; this section will be updated with that detail as it becomes available.

10b. Performance

BOTTOM LINE — FOR A 10-PAGE DOCUMENT

~6 LLM calls, ≈ \$0.05 per document on the provider actually serving calls today (Gemini 3.6 Flash), or ≈ \$0.03 on Nova once it's unblocked. At 1,000 such documents a month, that's roughly **\$30–50/month** — a small line item. Full breakdown, assumptions, and a worst-case figure follow below.

PENDING

Actual wall-clock latency and the token counts behind the figure above have not been measured yet for the graph pipeline — no run has logged them. The estimate above is grounded in the graph's real, configured chunking and retry limits (`app/graph/config.py`) plus real, current per-token pricing, but the token-size-per-call inputs are estimates, not measurements — see the note at the end of this section for the exact fix.

How many LLM calls does a ~10-page document take? Chunking is character-based, not page-based: a chunk splits at 24,000 characters, with 1-page overlap between chunks. A typical typed page runs roughly 1,500–3,000 characters, so a 10-page document lands as **1 chunk** (a sparser report) to **2 chunks** (a denser one). Of the 18 nodes, only 5 call an LLM, and only one of them — Dynamic Extraction — scales with chunk count; the other four (Document Analysis, Field Normalization, Conflict Resolution, Form Mapping) run once per document regardless of length.

Scenario	Chunks	LLM calls	How it's counted
Best case — clean extraction, no retries	1	5	1 analysis + 1 extraction + 1 normalization + 1 conflict-resolution + 1 form-mapping
Typical — clean extraction, denser document	2	6	Same as above, with 2 extraction calls (one per chunk) instead of 1
Worst case — every retry and reanalysis limit hit	2	~16	4 one-time calls + up to 6 extraction attempts (2 chunks × the configured 3-attempts-per-chunk retry ceiling) + up to 6 more from reanalysis (the configured 2 reanalysis rounds, each able to retry up to 3 times)

Worked cost example for a 10-page document. No run has logged real token counts yet, so the token sizes below are estimates (clearly marked), not measurements. The per-token prices are not estimates — current published rates as of September 2026, cited at the end of this section.

Call	Runs	Est. input tokens	Est. output tokens
Document Analysis	1	6,000	300
Dynamic Extraction	2 (one per chunk)	7,500 each	2,000 each
Field Normalization	1	2,500	1,500
Conflict Resolution	1	1,500	800
Form Mapping	1	3,500	1,800
Typical total (6 calls)	—	~28,500	~8,400
Worst-case total (~16 calls)	—	~103,600	~28,400

Provider	Input \$/1M tok	Output \$/1M tok	Typical doc (6 calls)	Worst-case doc (~16 calls)
Nova 2 Lite (configured primary — currently blocked, see below)	\$0.30	\$2.50	≈ \$0.03	≈ \$0.10
Gemini 3.6 Flash (what's actually serving calls right now)	\$0.75	\$3.75	≈ \$0.05	≈ \$0.18

At 1,000 documents a month, that's roughly **\$30–\$50/month** at the typical case, or **\$100–\$180/month** if every document hit every retry ceiling — on either provider, this is a small line item, not a cost driver. The bigger uncertainty is the token-size assumptions above, not the pricing: the fix for that is logging real `input_tokens / output_tokens` per call (Bedrock's Converse API already reports these — see `app/documents/call_budget.py`'s `TokenUsage`, currently only wired up for the legacy pipeline, not this graph) rather than re-deriving it from config limits.

Nova (the configured primary, `us.amazon.nova-2-lite-v1:0`) is currently blocked account-wide (verified directly: every call returns `ValidationException – "Operation not allowed"`, despite valid AWS credentials), so real spend right now is on the Gemini/Groq column, not the Nova column. Pricing sources: [Nova 2 Lite — Bedrock pricing](#); [Gemini 3.6 Flash — API pricing](#) (introductory rate through Dec 31, 2026; rises to \$1.50/\$7.50 per 1M tokens on Jan 1, 2027).

10c. Edge cases & known gaps

- The current benchmark/scoring scripts expect the labelled dataset laid out as one subfolder per document; the dataset was since reshaped into a flat directory of files. Run as-is today, the scripts would silently score zero documents rather than raising an error — this needs fixing before any new benchmark run.
- No bounding-box output from the graph pipeline's extraction step.
- One component's cloud entitlement (AWS Nova) is currently blocked, which the pipeline's own documentation notes as a reason no production accuracy figure exists yet.

- DOCX page numbering is approximated rather than reflecting true page breaks.
- A field-merge pass is skipped once a document produces more than 250 extracted field names.
- Zero water or soil documents exist in the most recent labelled evaluation set — those domains are effectively unvalidated.
- The legacy pipeline integration in this repository has been observed broken and re-applied more than once during recent development — worth treating the current live state as correct-but-fragile rather than settled.

10d. Glossary

Term	Meaning
NABL	National Accreditation Board for Testing and Calibration Laboratories — the body labs seek accreditation from; some documents in scope are accreditation application materials for NABL specifically.
OCR	Optical character recognition — converting a scanned image or photograph of a document into machine-readable text.
Domain	The category of lab report a document belongs to (medical, milk, food, water, soil, chemical, or other).
Field / key extraction	Pulling out document metadata as key-value pairs (e.g. patient name, sample ID, report date) rather than as a table row.
Ground truth	The hand-labelled, human-verified correct answer for a document, used to measure extraction accuracy.
Precision / Recall / F1	Precision: of what the system produced, how much was right. Recall: of what should have been produced, how much was it found. F1: the balance of the two into one number.
Graph pipeline	The current multi-node extraction architecture (§7b, Design B), where each processing step is a node in a directed graph rather than one monolithic call.
Checkpointing	Saving pipeline state after each step so processing can resume from that point rather than restarting from scratch.
Completeness check / targeted re-analysis	A step that detects an incomplete extraction and re-runs extraction focused on just the missing parts, instead of accepting a partial result.
Conflict resolution	Reconciling two different values found for what should be the same field, typically across different pages of one document.
Form mapping / form filling	Taking extracted, normalized data and placing it into the actual fields of a target NABL application form.
Human-in-the-loop	An explicit step where a person reviews and can correct the system's output before it's treated as final.